

ML CV

Summary

- Built phishing detection model for AI Cyber Threat Prevention app.
- Trained using text and URL features, including the PhiUSIIL dataset.
- Implemented probability calibration and threshold tuning for real-world scoring.

Skills

- - pandas – dataset I/O and preparation
- joblib – save/load trained model artifacts
- scikit-learn – core ML (HashingVectorizer, SGDClassifier, CalibratedClassifierCV, Pipeline, train_test_split, metrics)
- statistics (mean) – aggregate confidence in smoke tests
- urllib.parse (urlparse) – URL parsing for feature engineering
- re – regex for URL/domain extraction and heuristics
- json – persist model metadata (threshold, metrics)
- - HashingVectorizer, SGDClassifier, CalibratedClassifierCV
- - Feature engineering for URLs and phishing text
- - Model persistence with joblib and JSON metadata

Experience

- Developed `PhishingModelManager` to load or retrain models automatically.
- Added training logic for custom dataset input and PhiUSIIL schema support.
- Improved URL extraction for domain-only phishing links.
- Tuned live decision threshold for higher sensitivity while maintaining precision.

Key impact

- Increased phishing recall on real dataset samples.
- Protected users by catching suspicious URLs and enabling auto-quarantine.
- Created a reusable training workflow for future dataset swaps.

ML Code Examples

Model training and calibration

```
python
base_pipeline = Pipeline([
    (
        "vect",
        HashingVectorizer(
            lowercase=True,
            analyzer="char_wb",
            ngram_range=(3, 5),
            n_features=2**18,
            alternate_sign=False,
            norm="l2",
        ),
    ),
```

```

    ),
    (
        "clf",
        SGDClassifier(
            loss="log_loss",
            alpha=1e-5,
            max_iter=50,
            tol=1e-3,
            class_weight="balanced",
            random_state=42,
        ),
    ),
])
calibrated = CalibratedClassifierCV(base_pipeline, cv=5,
method="sigmoid")
calibrated.fit(x_train, y_train)

```

Feature construction for URLs

```

python
def build_feature_text(url: str = "", domain: str = "", tld: str = "",
title: str = "", raw_text: str = "") -> str:
    url = _safe_str(url).lower()
    domain = _safe_str(domain).lower()
    tld = _safe_str(tld).lower()
    title = _safe_str(title).lower()
    raw_text = _safe_str(raw_text).lower()

    parsed = urlparse(url) if url else urlparse(raw_text if
raw_text.startswith("http") else "")
    host = parsed.hostname or domain
    host = host.lower()
    tld_guess = tld or (host.split(".")[ -1] if "." in host else "")
    path_part = parsed.path or ""
    query = parsed.query or ""

    pieces = [raw_text, url, domain, tld_guess, host, title]
    pieces += [
        f"is_https_{int(parsed.scheme == 'https')}",
        f"is_ip_{int(bool(_URL_IP_RE.match(host)))}",
        f"url_len_{len(url)}",
        f"domain_len_{len(domain or host)}",
        f"host_len_{len(host)}",
        f"tld_len_{len(tld_guess)}",
        f"subdomains_{max(host.count('.') - 1, 0)}",
        f"path_len_{len(path_part)}",
        f"query_len_{len(query)}",
        f"has_percent_{int('%' in url)}",
        f"has_dash_{int('-' in host)}",
    ]
    return " ".join(p for p in pieces if p)

```

Screenshot:

```
base_pipeline = Pipeline([
    ("vect", HashingVectorizer(
        lowercase=True,
        analyzer="char_wb",
        ngram_range=(3, 5),
        n_features=2**18,
        alternate_sign=False,
        norm="l2",
    )),
    ("clf", SGDClassifier(
        loss="log_loss",
        alpha=1e-5,
        max_iter=50,
        tol=1e-3,
        class_weight="balanced",
        random_state=42,
    )),
])
```

```
def build_feature_text(url="", domain="", tld="", title="", raw_text=""):
    parsed = urlparse(url) if url else urlparse(raw_text if raw_text.startswith('http') else raw_text)
    host = (parsed.hostname or domain).lower()
    pieces = [raw_text.lower(), url.lower(), host]
    pieces += [
        f"is_https_{int(parsed.scheme == 'https')}",
        f"is_ip_{int(bool(_URL_IP_RE.match(host)))}",
        f"url_len_{len(url)}",
        f"has_dash_{int('-' in host)}",
    ]
    return " ".join(p for p in pieces if p)
```